GROUP ASSIGNMENT

TECHNOLOGY PARK MALAYSIA

CT077-3-2 DSTR

DATA STRUCTURES

APD2F2311CS(DA)

HAND OUT DATE: 13 May 2024

HAND IN DATE: 15 July 2024

WEIGHTAGE: 15%

INSTRUCTIONS TO CANDIDATES:

1. **Submit your assignment at the Moodle System.**
2. **Students are advised to underpin their answers with the use of references (cited using the APA Style System of Referencing).**
3. **Late submission will be awarded zero (0) unless Extenuating Circumstances (EC) are upheld.**
4. **Cases of plagiarism will be penalized.**
5. **The assignment should be submitted in softcopy, where the softcopy of the written assignment and source code (where appropriate) should be on Moodle System.**
6. **You must obtain 50% overall to pass this module**

Group No: 49

Ooi Chong Ming: TP072667

Yeoh Zi Qing Bryan: TP072717

Sim Sau Yang: TP065596

Table of Content

1.0 Introduction 1

2.0 Theoretical Study 1

2.1 Sorting Algorithm - Merge Sort 1

2.1.1 Implementation 1

2.1.2 Performance and Complexity 2

2.1.3 Advantages and Disadvantages 3

2.2 Sorting Algorithm – Quick Sort 3

2.2.1 Implementation 3

2.2.2 Performance and Complexity 6

2.2.3 Advantages and Disadvantages 6

2.3 Sorting Algorithm – Heap Sort 6

2.3.1 Implementation 7

2.3.2 Performance and Complexity 8

2.3.3 Advantages and Disadvantages 9

2.4 Sorting Algorithm – Selection Sort 9

2.4.1 Implementation of Selection Sort 9

2.4.2 Performance and Complexity 11

2.4.3 Advantages and Disadvantages 12

2.5 Sorting Algorithm – Insertion Sort 12

2.5.1 Implementation of Insertion Sort 12

2.5.2 Performance and Complexity 14

2.5.3 Advantages and Disadvantages 14

2.6 Sorting Algorithm Comparison 15

2.6.1 Comparison 15

2.6.2 Justification and Hypotheses 15

3.0 Comparative Experiment Implementation 16

3.1 Data Review 16

3.2 Building Structure 17

3.3 Reading and Processing Dataset 17

3.4 Sorting Algorithm Execution 20

3.4.1 Quick Sort 20

3.4.2 Heap Sort 20

3.4.3 Application of Sort Method 21

3.5 Result 22

3.5.1 CSV File Output (Heap Sort) 22

3.5.2 CSV File Output (Quick Sort) 23

3.5.3 Time Complexity Output 24

3.5.4 Observation 24

4.0 Discussion 24

5.0 Conclusion 25

6.0 Workload Matrix 26

7.0 References 27

Table of Figure

Figure 1: Merge Sort Example (Programiz, n.d.-b) 1
Figure 2: Selecting a pivot element 3
Figure 3: Comparison of pivot element with element beginning from the first index 4
Figure 4: Second Pointer 4
Figure 5: Comparison of pivot with other elements and swapping of elements 4
Figure 6: Process repeat 4
Figure 7: End of process 5
Figure 8: Swapping of pivot element with second pointer 5
Figure 9: Sorting elements on the left of the pivot using recursion 5
Figure 10: Sorting elements on the right of the pivot using recursion 5
Figure 11: Example of Max Heap and Min Heap 7
Figure 12: Example of unsorted array 7
Figure 13: Heapify (Max-Heap) 7
Figure 14: Second heapify 8
Figure 15: Result after first iteration 8
Figure 16: Final result of heap sort 8
Figure 17: First Pass of Selection Sort Algorithm (GeeksforGeeks, 2024) 10
Figure 18: Second Pass of Selection Sort Algorithm (GeeksforGeeks, 2024) 10
Figure 19: Third Pass of Selection Sort Algorithm (GeeksforGeeks, 2024) 10
Figure 20: Fourth Pass of Selection Sort Algorithm (GeeksforGeeks, 2024) 10
Figure 21: Final Pass of Selection Sort Algorithm (GeeksforGeeks, 2024) 11
Figure 22: Working of Insertion Sort Algorithm (GeeksforGeeks, 2024) 13
Figure 23: Apartment Structure 17
Figure 24: Read and Process Dataset 17
Figure 25: Read CSV line by line 17
Figure 26: parseCSVLine function 18
Figure 27: Ensure enough fields 18
Figure 28: Parse monthly rent 18
Figure 29: Parse other fields 19
Figure 30: Parse size 19

Figure 31: Clean “furnished” column 19
Figure 32: Partition function 20
Figure 33: Quick Sort function 20
Figure 34: Heap Sort function 20
Figure 35: Heapify function 21
Figure 36: Application of Quick Sort 21
Figure 37: CSV File Output: Heap Sort (Monthly Rent) 22
Figure 38: CSV File Output: Heap Sort (Size) 23
Figure 39: CSV File Output: Quick Sort (Monthly Rent) 23
Figure 40: CSV File Output: Quick Sort (Size) 23

1.0 Introduction

In order to support the rent system created for Asia Pacific Home (APH), this report examines a number of sorting algorithms and argues for the optimal sorting algorithm to fit the mudah-apartment-kl-selangor.csv dataset, which includes details of over 10,000 properties with rental pricing surrounding Kuala Lumpur and Selangor. The implementation of sorting algorithms will be accomplished by C++.

2.0 Theoretical Study

2.1 Sorting Algorithm - Merge Sort

Combine Sort utilises the divide and conquer algorithm concept and is a comparison-based sorting algorithm (Programiz, n.d.-b). The method divides the unsorted list into smaller sublists until there is only one element remaining in each sublist, and this element is naturally sorted (GeeksforGeeks, 2024c). After that, these sublists are combined once more to create a sorted list.

2.1.1 Implementation

The execution of merge sort involves the steps of divide, conquer, and merge (Khan Academy, n.d.). The list is split into two halves recursively during the divide stage until each sublist has a single entry. Since each sublist only has one entry in the conquer stage, they are then all regarded as sorted by default. The sublists are then combined once more in the order that they were sorted. The smallest components from each sublist are compared and combined into a new sorted list during the merging stage.

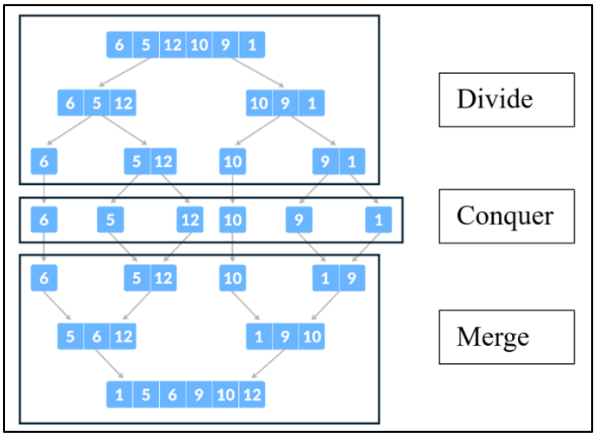

Figure 1: Merge Sort Example (Programiz, n.d.-b)

Figure 1 illustrates the merge sort process on the array {6, 5, 12, 10, 9, 1}. The array is repeatedly divided in half until each sub-array contains only one element, which is sorted by default. During the merge phase, the sub-arrays are combined and sorted. For instance, the sub-array {6, 5, 12} is divided into {6} and {5, 12}, and {5, 12} is further divided into {5} and {12}. When merging, pointers compare elements from the sub-arrays and place the smaller element into the parent array, resulting in {5, 12}. Finally, {6} and {5, 12} are merged by comparing 6 with 5, placing 5 first, followed by 6 and 12, resulting in {5, 6, 12}.

2.1.2 Performance and Complexity

Type	Complexity
Time Complexity	Best: O(n log n) Average: O(n log n) Worst: O(n log n)
Space Complexity	Worst: O(n)

Table 1: Complexity of Merge Sort (GeeksforGeeks, 2024c)

2.1.2.1 Time Complexity

With a time complexity of O(n log n), Merge Sort consistently outperforms average, worst-case, and best-case circumstances (Pankaj, 2022). This efficiency stems from the fact that Merge Sort consistently splits the list into halves, resulting in a logarithmic number of division levels (log n levels) and requires linear time to merge the sub lists at each level (DSA Merge Sort, n.d.). The logarithmic component is due to the repeated halving of the list, while the linear component is due to the merging operation. As a result, Merge Sort maintains this performance regardless of the initial order of the elements.

2.1.3 Advantages and Disadvantages

Advantages	Disadvantages
• Propose stable sorting as it maintains the relative order of equal elements • The O(n log n) time complexity is consistent and reliable across different input types, making Merge Sort suitable for large datasets. • Merge Sort is particularly well-suited for linked lists, where the additional memory overhead for temporary arrays can be minimized, and element swapping is not necessary	• The O(n) space complexity of merge sort requires additional memory • Merge Sort is slower for small arrays due to the overhead of recursive calls and the merging process

Table 2: Advantages and Disadvantages of Merge Sort (GeeksforGeeks, 2024c)

2.2 Sorting Algorithm – Quick Sort

Quicksort is renowned for its efficiency, working by dividing a large data array into two smaller arrays based on a pivot value. One array contains values greater than the pivot, and the other contains values less than the pivot *(Simplilearn, 2024).* Once the quicksort algorithm is applied, the elements greater than the pivot end up on its right, while those smaller are on its left. This process is recursively repeated on the left and right sub-arrays until each sub-array has only one element. Once sorted, these sub-arrays are merged to form the final sorted array *(QuickSort (With Code in Python/C++/Java/C), n.d.).*

2.2.1 Implementation

Step 1 – Select the Pivot Element

The pivot may appear as the first, last, middle, or element at random. A popular strategy is to use the final piece as the pivot.

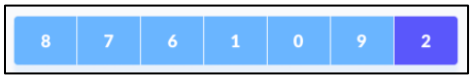

Figure 2: Selecting a pivot element

In Figure 2 shown above, an array example is shown. As shown in the Figure, the array's rightmost element, 2, will serve as the pivot element.

Step 2 – Rearrange the array

The array is then reorganized such that all elements smaller than the pivot are on the left, and all elements larger than the pivot are on the right. At this point, the pivot element is in its final sorted position. The steps to rearrange the elements are as follows:

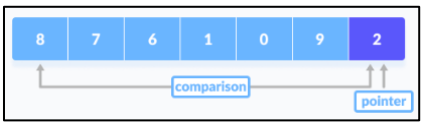

Figure 3: Comparison of pivot element with element beginning from the first index

A pointer is fixed to the pivot element in the first step of the rearranging procedure, as shown in Figure 3. The first index element and the pivot element are then compared after this step.

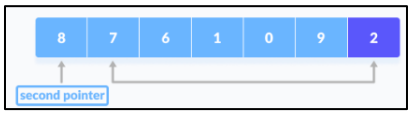

Figure 4: Second Pointer

As shown in Figure 4 above, if the element the pivot is comparing to is greater than the pivot element, a second pointer is set for that element.

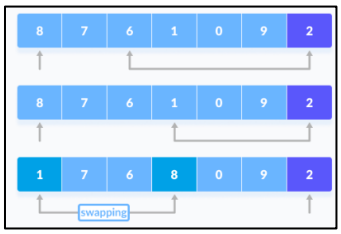

Figure 5: Comparison of pivot with other elements and swapping of elements

The pivot element is continuously compared with other elements. The larger element that was previously found will take the place of the smaller element when the value of the element comparing is less than the pivot element.

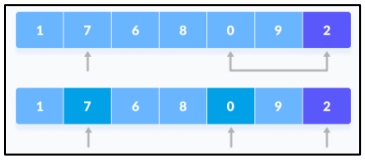

Figure 6: Process repeat

Following the swapping procedure, the next larger element of the swapped element is set as the second pointer, and when another smaller element is founded, the process is repeated to swap it with it.

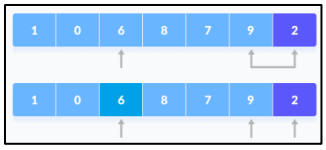

Figure 7: End of process

As Figure 7 shows, the process is repeated until the second last element is reached.

Figure 8: Swapping of pivot element with second pointer

The last step in the rearrangement process will be to switch the pivot element with the second pointer. After this point, the final sorted position will be represented by the items with values less than the pivot on the left side of the pivot and elements with values greater than the pivot on the right.

Step 3 –Recursive Sort

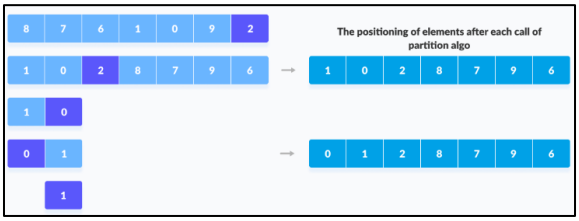

Figure 9: Sorting elements on the left of the pivot using recursion

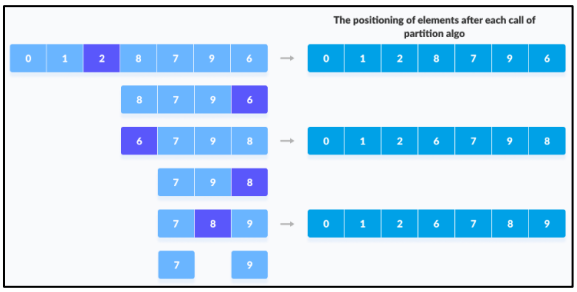

Figure 10: Sorting elements on the right of the pivot using recursion

Step 2 is repeated when pivot elements are once more selected for the left and right sub-parts independently, as seen in Figures 9 and 10. Recursive sort refers to this recursive application of step 2 to the left and right sub-arrays. This procedure keeps going until the array reaches the base case, which is when it has one or no elements, indicating that it has already been sorted.

2.2.2 Performance and Complexity

Type	Complexity
Time Complexity	Best scenario: $O(n \log n)$ Average scenario: $O(n \log n)$ Worst scenario: $O(n^2)$
Space Complexity	$O(\log n)$

Table 3:Complexity of Quick Sort (QuickSort (With Code in Python/C++/Java/C), n.d.)

2.2.2.1 Time Complexity

Quick Sort typically has a time complexity of $O(n\ log\ n)$, where n represents the number of elements in the array. This is because each partition step generally divides the array into two equal halves, requiring log n steps to completely break down the array *(Quick Sort - Javatpoint, n.d.).* However, in the worst-case scenario, Quick Sort's time complexity can degrade to O(n^2). This happens when the pivot selection is poor, such as choosing the largest or smallest element in an already sorted array. To mitigate this, effective pivot selection strategies, like randomly choosing an element or selecting the median of three elements, can be employed. Even under optimal conditions, where the array is evenly divided by the pivot, the time complexity remains $O(n\ log\ n)$ *(Quick Sort - Javatpoint, n.d.).*

2.2.3 Advantages and Disadvantages

Advantages	Disadvantages
• The divide-and-conquer approach aids in problem-solving. • It is efficient with large datasets. • It requires minimal memory, resulting in low overhead.	• If the pivot is poorly chosen, the worst-case time complexity is$O(n^2)$. • Not ideal for small datasets. • Does not preserve the relative order of equal elements, as it is not a stable sort.

Table 4: Advantages and Disadvantages of Quick Sort (GeeksforGeeks, 2024)

2.3 Sorting Algorithm – Heap Sort

Heap sort is a comparison-based sorting algorithm that operates on a binary heap data structure, maintaining an $O(n\ log\ n)$ time complexity. The array elements are represented as a heap, which is a complete binary tree *(Heap Sort (With Code in Python, C++, Java, and C), n.d.).* The process begins by creating a max-heap or min-heap from the input data, then repeatedly extracting the

maximum or minimum element and rebuilding the heap until all elements are sorted. This transformation is achieved using the heapify method, which recursively heapifies each node (Heap Sort Algorithm, n.d.). A binary tree follows a heap structure if it forms a max-heap, where nodes are larger than their children, or a min-heap, where nodes are smaller *(Heap Sort (With Code in Python, C++, Java, and C), n.d.).* The final result is an array sorted in ascending order with a max-heap or in descending order with a min-heap.

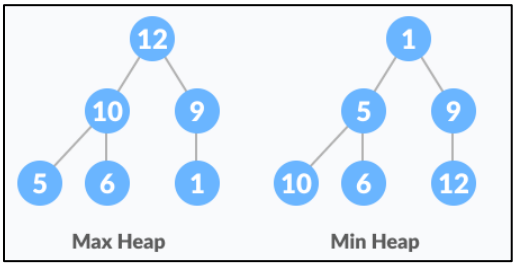

Figure 11: Example of Max Heap and Min Heap

2.3.1 Implementation

In the following example, we will show an example by using max-heap.

81	89	9	11	14	76	54	22

Figure 12: Example of unsorted array

An illustration of the unsorted array we'll be using in this section is shown in Figure 12 above.

Step 1 – Heapify (Build a Max-Heap)

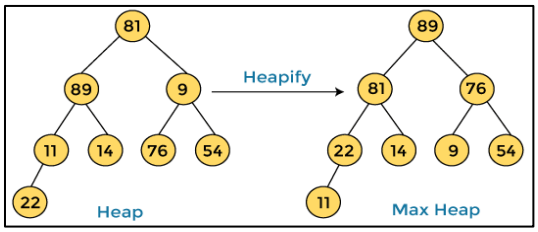

Figure 13: Heapify (Max-Heap)

The heapify approach, depicted in Figure 13 above, turns the unsorted binary tree into a max heap in which every node is larger than any of its offspring nodes.

Step 2 – Swap and Delete

The next step is to remove the max heap's root element, in this case 89, from the max heap. We must first swap this node with the final node—in our case, node 11—in order to delete it. To make it into max heap, we must heapify it once more after removing the root element.

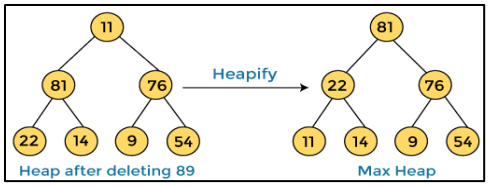

Figure 14: Second heapify

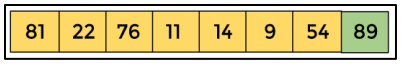

Figure 15: Result after first iteration

The second heapify procedure is depicted in Figure 14 above after the root element (89) is removed. Figure 15 also displays the outcome of our first iteration, which demonstrates that the largest element has been positioned correctly and that the root element is now at the end of the array.

Step 3 – Repeat Application of Step 2

After the first iteration, step 2 is repeated for the result binary tree from the precious iteration until the binary tree contains one element left, which should be the smallest element in the array.

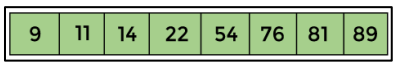

Figure 16: Final result of heap sort

Figure 16 above shows the final result of the heap sort application, where the unsorted array had been successfully sorted into ascending order.

2.3.2 Performance and Complexity

Type	Complexity
Time Complexity	Best scenario: $O(n \log n)$ Average scenario: $O(n \log n)$ Worst scenario: $O(n \log n)$
Space Complexity	$O(1)$

Table 5:Complexities of Heap Sort (Heap Sort - Javatpoint, n.d.)

2.3.2.1 Time Complexity

Heap sort's time complexity is always $O(n \log n)$ in every situation. This is due to the fact that each extraction and heapify process takes $O(\log n)$time, whereas it takes $O(n)$ time to create the heap from the input data. Because there are n elements to extract, $O(n \log n)$ is the overall complexity. The array elements that are jumbled and do not follow the standard ascending or descending order result in the average case complexity, whereas the best-case complexity arises

when sorting is not necessary. However, having to sort the array's members in reverse order results in the worst-case complexity. An example of worst-case scenario is arranging an array's components ascendingly, when the array itself is originally in descending order (*Heap Sort - Javatpoint*, n.d.).

2.3.3 Advantages and Disadvantages

Advantages	Disadvantages
• Efficient time complexity - efficient for sorting large datasets as it maintains good performance even with a large number of elements. • Memory usage can be minimal – no additional memory space is needed. • Simplicity - simpler to understand than other equally efficient sorting algorithms	• Costly - constants are higher compared to other algorithm even if the time complexity is $O(n \log n)$ for both. • Unstable - relative order of equal elements is not maintained. • Efficiency - not very efficient when working with highly complex data.

Table 6: Advantages and Disadvantages of Heap Sort (GeeksforGeeks, 2024)

2.4 Sorting Algorithm – Selection Sort

Selection sort is a straightforward and efficient algorithm that arranges a list by repeatedly choosing the smallest (or largest) element from the unsorted section and moving it to the beginning of the sorted section. *(Selection Sort Algorithm, 2014).* According to *(The Selection Sort Algorithm Explained with Examples, 2019)*, there will be 2 subarrays which is sorted subarray and unsorted subarray. In each iteration of the selection sort algorithm, the smallest (or largest) element from the unsorted subarray is selected and swapped with the first element of that subarray. This process continues with the remaining unsorted sections until the entire list is sorted.

2.4.1 Implementation of Selection Sort

The selection sort algorithm involves two subarrays: one sorted and one unsorted. It is simple and effective for small datasets. The algorithm works by repeatedly finding the smallest (or largest) element from the unsorted part of the array and moving it to the sorted part. Here is how it operates, step-by-step, using the given examples:

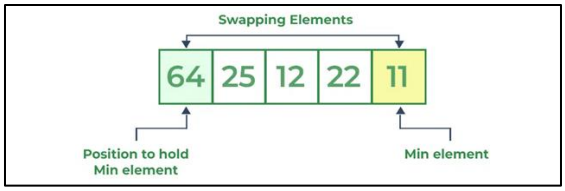

Figure 17: First Pass of Selection Sort Algorithm (GeeksforGeeks, 2024)

Consider the array [64, 25, 12, 22, 11] with index from 0 to 4. To sort in ascending order, 11 is the smallest element and is swapped with the first element (64), resulting in [11, 25, 12, 22, 64]. Now, 11 is correctly positioned at the start of the sorted section.

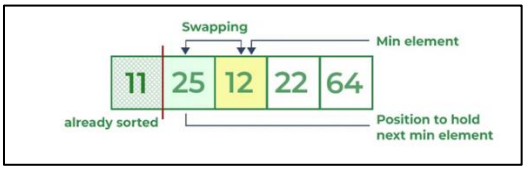

Figure 18: Second Pass of Selection Sort Algorithm (GeeksforGeeks, 2024)

In the second step, the algorithm moves to the second position and scans the remaining unsorted portion of the array ([25, 12, 22, 64]). It identifies 12 as the smallest value and swaps it with the second element (25), updating the array to [11, 12, 25, 22, 64]. Now, 12 is correctly positioned.

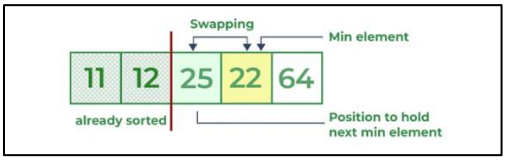

Figure 19: Third Pass of Selection Sort Algorithm (GeeksforGeeks, 2024)

The algorithm proceeds to the third position, examining the unsorted portion [25, 22, 64]. It finds 22 to be the smallest value and swaps it with the third element (25), resulting in [11, 12, 22, 25, 64]. Now, 22 is correctly positioned.

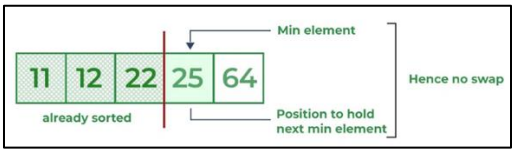

Figure 20: Fourth Pass of Selection Sort Algorithm (GeeksforGeeks, 2024)

Similarly, in the fourth step, the algorithm focuses on the fourth position and scans the remaining unsorted portion [25, 64]. It finds that 25 is the fourth smallest value and swaps it with the fourth element, leaving the array as [11, 12, 22, 25, 64]. Here, 25 is correctly positioned.

Figure 21: Final Pass of Selection Sort Algorithm (GeeksforGeeks, 2024)

In the final pass, only one element remains unsorted, which is 64. Since it is the largest value, it is already in its correct position. Thus, the array remains [11, 12, 22, 25, 64], resulting in a fully sorted array.

Through this method, selection sort ensures that with each iteration, the smallest element from the unsorted section is placed in its final position in the sorted section. This step-by-step selection and swapping make selection sort a simple yet clear example of how sorting algorithms can organize data efficiently.

2.4.2 Performance and Complexity

Type	Complexity
Time Complexity	Best scenario: $O(n^2)$ Average scenario: $O(n^2)$ Worst scenario: $O(n^2)$
Space Complexity	$O(1)$

Table 7: Complexities of Selection Sort (Time and Space Complexity Analysis of Selection Sort, 2023)

2.4.2.1 Time Complexity

According to *(Time and Space Complexity Analysis of Selection Sort, 2023),* selection sort's time complexity is $O(n^2)$ in all cases (best, average, and worst). As *(Vaia, n.d.)* states, selection sort is valued for its efficiency with smaller datasets and its simple, easy-to-understand implementation. Despite its $O(n^2)$ time complexity making it less ideal for larger arrays, it remains a popular teaching tool for illustrating fundamental concepts in computational logic and algorithmic thinking. The initial order of the input array does not impact the time complexity; in each pass, the algorithm scans the entire array to find the minimum element and places it in the correct position in the sorted section.

2.4.3 Advantages and Disadvantages

Advantages	Disadvantages
• Simple and easy to implement. • In-place sorting, meaning it sorts the array directly without requiring additional storage. • Requires minimal swaps compared to other sorting algorithms like Bubble Sort, which is advantageous in scenarios where swap operations are costly.	• Selection sort has a time complexity of $O(n^2)$ in both the worst and average cases. • It performs inefficiently on large datasets. • It does not maintain the relative order of objects with equal keys, making it unstable.

Table 8: Advantages and Disadvantages of Selection Sort (Time and Space Complexity Analysis of Selection Sort, 2023)

2.5 Sorting Algorithm – Insertion Sort

Insertion sort is a straightforward and stable sorting algorithm described by (GeeksforGeeks, 2024). It arranges elements by iteratively inserting each item from an unsorted list into its correct position within a sorted section of the list. The algorithm operates in-place, meaning it sorts the array directly without requiring additional memory. Beginning with the second element, each element is compared with previous elements and swapped if necessary to ensure it is correctly positioned. This process continues until the entire array is sorted.

2.5.1 Implementation of Insertion Sort

To carry out insertion sort, follow these steps:

1. Begin with the second element of the array, assuming the first element is already sorted.
2. Compare the second element with the first. If the second element is smaller, swap them.
3. Move to the third element. Compare it first with the second element and then with the first element, making necessary swaps to insert it correctly among the first three elements.
4. Repeat this process for each subsequent element. Compare each new element with the elements before it, swapping it as needed to place it correctly within the sorted section of the array.
5. Continue this method until the entire array is sorted.

2.5.1.1 Example of implementation using array (GeeksforGeeks, 2024):

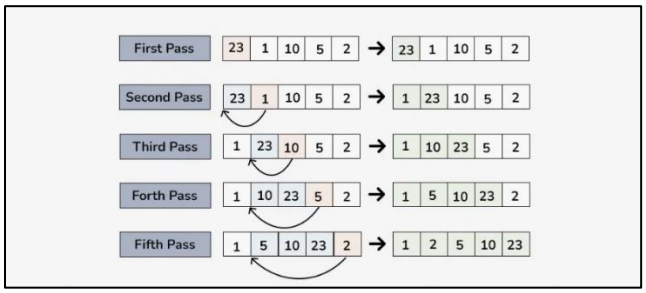

Figure 22: Working of Insertion Sort Algorithm (GeeksforGeeks, 2024)

Considering an array with elements [23, 1, 10, 5, 2], let's walk through the Insertion Sort algorithm.

- **First Pass**: The initial element is 23. Since we assume the first element is already sorted, the sorted section up to index 0 is: [23].
- **Second Pass**: The current element is 1. We compare 1 with 23. As 1 is smaller, we insert 1 before 23. The sorted section up to index 1 is: [1, 23].
- **Third Pass**: Right now, there are ten elements. We compare 10 with [1, 23], the sorted part. We position 10 between 1 and 23 as we know it is larger than 1 but smaller than 23. [1, 10, 23] is the sorted portion up to index 2.
- **Fourth Pass**: The current element is 5. We compare 5 with the sorted section, [1, 10, 23]. We then insert 5 between 1 and 10 as 5 is larger than 1 but smaller than 10. [1, 5, 10, 23] is the sorted index 3.
- **Fifth Pass**: The current element is 2. We compare 2 with the sorted section, [1, 5, 10, 23]. We position 2 between 1 and 5 since it is greater than 1 but less than 5. This is the sorted section [1, 2, 5, 10, 23] up to index 4.

After completing all passes, the array is fully sorted: [1, 2, 5, 10, 23].

Insertion Sort is a straightforward algorithm suitable for small or nearly sorted datasets. It sorts by comparing and inserting each element into its correct position one at a time. Despite its worst-case time complexity of $O(n^2)$, the algorithm is stable, ensuring it maintains the relative order of equal elements. This stability is advantageous in various applications.

2.5.2 Performance and Complexity

Type	Complexity
Time Complexity	Best scenario: $O(n)$ Average scenario: $O(n^2)$ Worst scenario: $O(n^2)$
Space Complexity	$O(1)$

Table 9: Performance and Complexity of Insertion Sort (Code 360 by Coding Ninjas, n.d.)

2.5.2.1 Time Complexity

Insertion Sort is a straightforward algorithm that arranges a list by placing each item in its correct position, similar to sorting cards in your hand. Its efficiency varies based on the initial state of the array. In the best-case scenario, where the array is already sorted, Insertion Sort operates in linear time, $O(n)$, as each element is compared only with its predecessors until no swaps are necessary. This makes it highly efficient for nearly ordered data. However, in average and worst cases where the array is random or sorted in reverse, Insertion Sort's performance decreases significantly due to its quadratic time complexity of $O(n^2)$. In these scenarios, each element may need to be compared and potentially swapped with every preceding element, leading to a slower process proportional to the square of the array size. Therefore, while Insertion Sort is effective for small datasets or nearly sorted arrays, its efficiency diminishes as the size of the array and its disorder increase.

2.5.3 Advantages and Disadvantages

Advantages	Disadvantages
• Simple and straightforward to implement. • Stable sorting algorithm (preserves the relative order of equal elements). • Efficient for small lists and nearly sorted lists. • Space-efficient (operates in-place).	• Inefficient for large lists (quadratic time complexity, $O(n^2)$). • Less efficient compared to advanced sorting algorithms like merge sort and quick sort in most cases.

Table 10: Advantages and Disadvantages of Insertion Sort (GeeksforGeeks, 2024)

2.6 Sorting Algorithm Comparison

2.6.1 Comparison

	Time Complexity	Space Complexity
Merge Sort	$O(n \log n)$	$O(n)$
Insertion Sort	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(1)$
Quick Sort	$O(n \log n)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(1)$

Table 11: Comparison between six sorting algorithms

We have chosen Quick Sort and Heap Sort as the most effective sorting algorithms for the "Mudah Apartment KL Selangor" dataset. We opted not to use Insertion Sort, and Selection Sort because their time complexities are $O(n^2)$, making them inefficient for our dataset, which is larger and exceeds their optimal handling capabilities. While Merge Sort is stable and efficient with a time complexity of $O(n\ log\ n)$, it requires significant additional memory compared to Quick Sort and Heap Sort. In comparison, Quick Sort and Heap Sort offer efficient performance with $O(n\ log\ n)$ time complexity and require less additional memory, making them more suitable for our dataset.

2.6.2 Justification and Hypotheses

In this assignment, after evaluating the time and space complexities of the five sorting algorithms on the large "Mudah Apartment KL Selangor" dataset, we have chosen Quick Sort and Heap Sort as the most suitable options. Quick Sort is preferred for its overall better performance in practical applications, especially when careful pivot selection avoids worst-case scenarios. On the other hand, Heap Sort is recommended for scenarios requiring guaranteed worst-case performance and minimal memory usage. These considerations make Quick Sort and Heap Sort ideal choices for efficiently sorting the dataset in this assignment.

To support our selection of Quick Sort and Heap Sort, we provided some hypotheses for the application of these two sorting algorithms in this assignment:

a) Because Quick Sort has fewer constant components than Heap Sort, it will probably perform better overall. It is perfect for practical application on huge datasets due to its average-case time complexity of $O(n \log n)$ with excellent pivot selection.

b) When worst-case performance is a major issue, heap sort is a reliable option since it guarantees consistent $O(n \log n)$ performance independent of the data's original order.

3.0 Comparative Experiment Implementation

3.1 Data Review

Column	Finding
ads_id	No problem found
prop_name	Contains missing values, some cells contain "," which need to be handled carefully when reading a CSV file.
completion_year	Contains missing values
monthly_rent	Contains missing values. The strings "RM" and "per month" should be removed (inconsistent). Monthly is suitable for sorting target
location	No problem found
property_type	No problem found
rooms	Contains missing values. The strings "more than" should be removed to remain only integers (inconsistent).
parking	Contains missing values
bathroom	Contains missing values
size	The strings "sq.ft." should be removed (inconsistent). Size is suitable for sorting target
furnished	Contains missing values
facilities	Meaningless attributes
additional_facilities	Meaningless attributes
region	Meaningless attributes

Table 12: Review Each Column

3.2 Building Structure

```
const int MAX_APARTMENTS = 19991; // Total observations in dataset

// Struct to represent an apartment
struct Apartment {
    string ads_id;
    string prop_name;
    string completion_year;
    int monthly_rent;
    string location;
    string property_type;
    int rooms;
    int parking;
    int bathroom;
    int size;
    string furnished;
};

// Global array to store apartments and a counter for the number of apartments
Apartment apartments[MAX_APARTMENTS];
int apartment_count = 0;
```

Figure 23: Apartment Structure

A new data type apartment is created to store all the attributes of the property. The facilities, additional facilities and region are not included.

3.3 Reading and Processing Dataset

```
int main() {
    string filename = "mudah-apartment-kl-selangor.csv";
    ifstream file(filename);
    string line;

    // Check if the file is opened successfully
    if (!file.is_open()) {
        cerr << "Failed to open file: " << filename << endl;
        return 1;
    }
    getline(file, line);  // Skip header row
```

Figure 24: Read and Process Dataset

To read the dataset, the input stream is used and the first column which is the header is skipped.

```
// Read the CSV file line by line
while (getline(file, line) && apartment_count < MAX_APARTMENTS) {
    string cells[11];
    parseCSVLine(line, cells, 11); // Parse the line into individual cells
    Apartment& apartment = apartments[apartment_count];
```

Figure 25: Read CSV line by line

Cell array stores elements of a row, and apartment array stores the apartment data type. Loop each element within every single row to add data to array.

```
void parseCSVLine(string line, string cells[], int max_cells) {
    string cell;                // Temporary string to store individual cells
    bool insideQuote = false;   // Indicate if the parser is inside a quoted string
    int cell_index = 0;         // Index to track the position in the `cells` array

    // Replace commas inside quotes with a placeholder
    for (size_t i = 0; i < line.size(); ++i) {
        if (line[i] == '"') {
            // Toggle the insideQuote flag when a quote is encountered
            insideQuote = !insideQuote;
        } else if (line[i] == ',' && insideQuote) {
            line[i] = '=';  // Replace comma with placeholder
        }
    }
    // Use a stringstream to split the line by commas
    stringstream ss(line);
    while (getline(ss, cell, ',') && cell_index < max_cells) {
        // Replace placeholder back to comma
        replace(cell.begin(), cell.end(), '=', ',');
        // Store the cell in the cells array
        cells[cell_index++] = cell;
    }
}
```

Figure 26: parseCSVLine function

Each row is originally a string and comma-separated each element. In this case, the quotation mark is checked to determine that the comma next to the quotation mark is a part of the element, not a separator, otherwise, the output element will not align with its corresponding column.

```
// Ensure the line has enough fields
if (!cells[0].empty()) {
    // Populate the apartment struct with data from cells
    apartment.ads_id = cells[0];
    apartment.prop_name = !cells[1].empty() ? cells[1] : "N/A";
    apartment.completion_year = !cells[2].empty() ? cells[2] : "N/A";
```

Figure 27: Ensure enough fields

After each element of a row is added to the array, they are inserted into the apartment's attribute. The empty cell of property name and completion year are filled with "N/A".

```
// Parse monthly rent
if (!cells[3].empty()) {
    try {
        string rent_str = cells[3];
        size_t pos = rent_str.find("RM ");
        if (pos != string::npos) {
            rent_str.erase(0, pos + 3);  // Remove "RM "
            pos = rent_str.find(" per month");
            if (pos != string::npos) {
                rent_str.erase(pos);  // Remove " per month"
            }
            // Remove spaces from rent_str
            rent_str.erase(remove(rent_str.begin(), rent_str.end(), ' '), rent_str.end());
            if (isInteger(rent_str)) {
                apartment.monthly_rent = stoi(rent_str);
            } else {
                apartment.monthly_rent = 0;  // Default value if invalid
            }
        } else {
            apartment.monthly_rent = 0;  // Default value
        }
    } catch (const std::invalid_argument&) {
        apartment.monthly_rent = 0;  // Default value
    }
} else {
    apartment.monthly_rent = 0;  // Default value
}
```

Figure 28: Parse monthly rent

The missing value in monthly rent cannot be replaced using “N/A” as it will be used for sorting. Hence, it is replaced by 0. The string “RM” and “per month” are also removed. The value is then parsed to integer and stored into apartment monthly rent attribute.

```
// Parse other fields
apartment.location = cells[4];
apartment.property_type = cells[5];
apartment.rooms = isInteger(cells[6]) ? stoi(cells[6]) : 0;  // Default value
apartment.parking = isInteger(cells[7]) ? stoi(cells[7]) : 0;  // Default value
apartment.bathroom = isInteger(cells[8]) ? stoi(cells[8]) : 0;  // Default value
```

Figure 29: Parse other fields

The location and property type do not have a problem and can be added directly. The rooms, parking, and bathroom are validated as integer before parsing them to an integer.

```
// Parse size
if (!cells[9].empty()) {
    try {
        string size_str = cells[9];
        size_t pos = size_str.find(" sq.ft.");
        if (pos != string::npos) {
            size_str.erase(pos);  // Remove " sq.ft."
        }
        apartment.size = isInteger(size_str) ? stoi(size_str) : 0;  // Default value
    } catch (const std::invalid_argument&) {
        apartment.size = 0;  // Default value
    }
} else {
    apartment.size = 0;  // Default value
}
```

Figure 30: Parse size

The handling way of size is similar to monthly rent, which replace the missing value with 0, and remove the string before parsing the element into an integer.

```
    apartment.furnished = !cells[10].empty() ? cells[10] : "N/A";
    // Increment the count of apartments
    apartment_count++;
```

Figure 31: Clean “furnished” column

The missing value in the furnished column is replaced by “N/A”. After the reading process is completed, the file stream closes.

3.4 Sorting Algorithm Execution

3.4.1 Quick Sort

```
int partitionMonthlyRent(Apartment arr[], int low, int high) {
    // Choose the last element as the pivot
    int pivot = arr[high].monthly_rent;
    int i = low - 1;    // Index of the smaller element

    // Loop through the array from `low` to `high - 1`
    for (int j = low; j <= high - 1; j++) {

        // If the current element is smaller than the pivot
        if (arr[j].monthly_rent < pivot) {
            i++;
            swap(arr[i], arr[j]); // Swap the current element with the element at index `i`
        }
    }

    // Swap the pivot element with the element at index `i + 1`
    swap(arr[i + 1], arr[high]);
    return i + 1;   // Return the index of the pivot element
}
```

Figure 32: Partition function

```
void quicksortMonthlyRent(Apartment arr[], int low, int high) {
    // Check if the low index is less than the high index
    if (low < high) {

        // Partition the array and get the pivot index
        int pi = partitionMonthlyRent(arr, low, high);

        // Recursively sort the elements before the pivot
        quicksortMonthlyRent(arr, low, pi - 1);

        // Recursively sort the elements after the pivot
        quicksortMonthlyRent(arr, pi + 1, high);
    }
}
```

Figure 33: Quick Sort function

Figure 33 above shows an example of quick sort for monthly rent, which is used for this part of explanation. The function first checks if the sub-array has more than one element to sort, and if so, it calls the partitionMonthlyRent function as shown in Figure 32 to determine the pivot's correct position. It then recursively sorts the sub-arrays before and after the pivot. The quick sort algorithm used to sort the size is similar to this example, where the only difference is that the parameter used is size instead of monthly rent.

3.4.2 Heap Sort

```
void heapsortMonthlyRent(Apartment arr[], int n) {
    // Build a max heap from the array
    for (int i = n / 2 - 1; i >= 0; i--)
        heapifyMonthlyRent(arr, n, i);

    // One by one extract elements from the heap
    for (int i = n - 1; i > 0; i--) {
        // Move the current root to the end
        swap(arr[0], arr[i]);

        // Call heapify on the reduced heap
        heapifyMonthlyRent(arr, i, 0);
    }
}
```

Figure 34: Heap Sort function

```
void heapifyMonthlyRent(Apartment arr[], int n, int i) {
    int largest = i;        // Initialize largest as root
    int left = 2 * i + 1;   // Left child index
    int right = 2 * i + 2;  // Right child index

    // If the left child exists and is greater than the root
    if (left < n && arr[left].monthly_rent > arr[largest].monthly_rent)
        largest = left;

    // If the right child exists and is greater than the largest so far
    if (right < n && arr[right].monthly_rent > arr[largest].monthly_rent)
        largest = right;

    // If the largest is not root
    if (largest != i) {
        // Swap the root with the largest
        swap(arr[i], arr[largest]);

        // Recursively heapify the affected subtree
        heapifyMonthlyRent(arr, n, largest);
    }
}
```

Figure 35: Heapify function

The example in Figure 34 illustrates heap sort applied to sorting by monthly rent. It begins by constructing a max heap from the array, ensuring each parent node is greater than its child nodes. This is achieved through the heapifyMonthlyRent function, which maintains the heap property for each subtree. Once the max heap is built, the algorithm repeatedly extracts the largest element (the root) and places it at the end of the array, reducing the heap size each time. After moving the root, heapifyMonthlyRent is called again to restore the heap property for the remaining elements. This process continues until the entire array is sorted in ascending order of monthly rent, effectively organizing the apartment objects. The heap sort algorithm used for sorting by size follows a similar approach, with the only difference being the parameter used (size instead of monthly rent).

3.4.3 Application of Sort Method

```
// Create a copy of the apartments array to avoid modifying the original array
Apartment* apartmentsCopy3 = new Apartment[apartment_count];
copy(apartments, apartments + apartment_count, apartmentsCopy3);

auto start3 = high_resolution_clock::now();
quicksortMonthlyRent(apartmentsCopy3, 0, apartment_count - 1);
auto stop3 = high_resolution_clock::now();
// Calculate the duration of the quick sort operation in milliseconds
auto duration3 = duration_cast<milliseconds>(stop3 - start3);

ofstream outFile3("QuickSort(Monthly_Rent)-Mudah-Apartment-KL-Selangor.csv");
outFile3 << header << endl;
for (int i = 0; i < apartment_count; i++) {
    Apartment& apartment = apartmentsCopy3[i];
    outFile3 << apartment.ads_id << ","
             << apartment.prop_name << ","
             << apartment.completion_year << ","
             << apartment.monthly_rent << ","
             << apartment.location << ","
             << apartment.property_type << ","
             << apartment.rooms << ","
             << apartment.parking << ","
             << apartment.bathroom << ","
             << apartment.size << ","
             << apartment.furnished << endl;
}
outFile3.close();
// Deallocate the memory used for the apartmentsCopy1 array
delete[] apartmentsCopy3;
cout << "Quick Sort (Monthly Rent) took " << duration3.count() << " milliseconds" << endl;
```

Figure 36: Application of Quick Sort

Figure 36 demonstrates the application of Quick Sort on the monthly rent parameter. Initially, a header for the sorted dataset output file is created. The "monthly rent" column has "RM" appended, and the "size" column has "sq.ft." appended to enhance dataset clarity after removing these strings from elements. To preserve the original data, a copy of the array is made before sorting begins. The start time of the sorting process is recorded using "high_resolution_clock::now()". The quicksortMonthlyRent function is then invoked to sort the copied array based on each apartment's monthly rent. Once sorting is finished, the end time is recorded, and the duration of the sorting operation in milliseconds is calculated. The sorted data, including apartment details like ads ID, property name, completion year, location, property type, rooms, parking, bathroom, size, and furnished status, is written to a CSV file. Finally, memory allocated for the copied array is freed, and the sorting operation duration is displayed on the console. This method provides an effective and systematic approach to sorting and documenting apartment data. The application of Heap Sort is same with the application of Quick Sort, just using different functions.

3.5 Result

3.5.1 CSV File Output (Heap Sort)

	A	B	C	D	E	F	G	H	I	J	K
1	ads_id	prop_name	completion_year	monthly_rent (RM)	location	property_type	rooms	parking	bathroom	size (sq.ft.)	furnished
2	100849484	N/A	N/A	0	Selangor - 149	Bungalow House	0	0	0	534	N/A
3	100849472	Affina Bay @ Butterworth	N/A	0	Selangor - 77	Condominium	0	0	0	1500	N/A
4	100549909	One South	2013	70	Selangor - Seri Kembangan	Service Residence	0	0	0	100	Fully Furnished
5	99410613	Setapak Ria Condominium	1994	80	Kuala Lumpur - Setapak	Condominium	1	0	0	5	Not Furnished
6	98342005	N/A	N/A	80	Selangor - Kota Damansara	Apartment	0	0	0	400	Fully Furnished
7	100676485	N/A	N/A	85	Selangor - Seri Kembangan	Soho	0	0	0	761	N/A
8	99974016	Idaman Sutera	N/A	90	Kuala Lumpur - Setapak	Condominium	1	0	0	100	Not Furnished
9	99846469	N/A	N/A	100	Kuala Lumpur - Mid Valley City	Residential	0	0	0	100	N/A
10	100578461	Kristal Heights (Shah Alam)	2006	100	Selangor - Shah Alam	Condominium	0	0	0	1000	Not Furnished
11	99389782	SD Tiara Apartment	N/A	100	Selangor - Bandar Sri Damansara	Apartment	0	0	0	400	Fully Furnished
12	100183866	Melur Apartment (Sentul)	2006	100	Kuala Lumpur - Sentul	Apartment	1	0	0	100	Not Furnished
13	100129853	Platinum Hill PV 2	2013	100	Kuala Lumpur - Setapak	Condominium	1	0	0	100	Not Furnished
14	100213071	Platinum Lake PV 15	N/A	100	Kuala Lumpur - Setapak	Condominium	4	0	0	1500	Fully Furnished
15	100163744	Platinum Lake PV8	N/A	100	Kuala Lumpur - Setapak	Condominium	1	0	0	1	Not Furnished
16	99872270	Platinum Lake PV 15	N/A	100	Kuala Lumpur - Setapak	Condominium	1	0	0	500	Not Furnished
17	100255897	Angkasa Condominiums	2005	110	Kuala Lumpur - Cheras	Condominium	1	0	0	99999999	Not Furnished
18	100694064	The Grand Subang SS13	N/A	120	Selangor - Subang Jaya	Service Residence	0	0	0	548	Fully Furnished
19	99974687	Pangsapuri Melur (Sentul)	2006	120	Kuala Lumpur - Sentul	Apartment	3	0	0	1000	Not Furnished
20	99955705	Riverville Residences	2015	120	Kuala Lumpur - Old Klang Road	Condominium	3	0	0	1350	Fully Furnished
21	97105324	Maxim Citilights	2017	120	Kuala Lumpur - Sentul	Service Residence	1	0	0	160	Not Furnished
22	100501147	Bayu Puteri	2004	120	Selangor - Petaling Jaya	Apartment	0	0	0	300	Partially Furnished
23	98295641	mont kiara pines	N/A	145	Kuala Lumpur - Mont Kiara	Condominium	1	0	0	120	Not Furnished
24	100627409	N/A	N/A	150	Selangor - Sungai Buloh	Others	0	0	0	25	Not Furnished
25	100221594	Southlink	N/A	150	Kuala Lumpur - Bangsar South	Apartment	1	0	0	60	Not Furnished
26	100810227	BSP21	2018	150	Selangor - Jenjarom	Service Residence	0	0	0	40	Not Furnished
27	100203686	Residensi Pandanmas 1	N/A	150	Kuala Lumpur - Desa Pandan	Apartment	1	0	0	100	Not Furnished
28	100230309	Platinum Hill PV 5	2006	150	Kuala Lumpur - Setapak	Condominium	1	0	0	80	Not Furnished
29	100775428	Amara Service Residences	2014	150	Selangor - Batu Caves	Service Residence	0	0	0	180	Not Furnished
30	100149093	N/A	N/A	150	Kuala Lumpur - Sentul	Condo / Services re	0	0	0	100	Not Furnished

Figure 37: CSV File Output: Heap Sort (Monthly Rent)

From Figure 37 above, we can clearly see that the output file's "Monthly Rent" attribute is already sorted from minimum to maximum. Meanwhile the "Size" attribute still remain unsorted, this means that we are using different array of container for all of the sorting processes.

	A	B	C	D	E	F	G	H	I	J	K
1	ads_id	prop_name	completion_year	monthly_rent (RM)	location	property_type	rooms	parking	bathroom	size (sq.ft.)	furnished
2	100618080	Laman Suria (Kajang)	N/A	850	Selangor - Kajang	Apartment	0	0	0	1	Not Furnished
3	100494809	Eclipse Residence @ Pan'gaea	2018	1200	Selangor - Cyberjaya	Service Residence	0	0	0	1	Fully Furnished
4	100653347	Kelana Sentral	N/A	2500	Selangor - Petaling Jaya	Condominium	0	0	0	1	Fully Furnished
5	100634656	Magna Ville Condominium	2010	950	Selangor - Selayang	Condominium	0	0	0	1	Not Furnished
6	98651627	N/A	N/A	2000	Kuala Lumpur - Cheras	Service Residence	1	0	0	1	Fully Furnished
7	99945149	Nidoz Residences	2019	180	Kuala Lumpur - Desa Petaling	Condominium	1	0	0	1	Not Furnished
8	100756915	Suria Jaya	N/A	1300	Selangor - Shah Alam	Apartment	0	0	0	1	Partially Furnished
9	100163744	Platinum Lake PV8	N/A	100	Kuala Lumpur - Setapak	Condominium	1	0	0	1	Not Furnished
10	100721981	Mahkota Garden Residence Condo	N/A	1600	Selangor - Bandar Mahkota Cheras	Condominium	0	0	0	2	Fully Furnished
11	100779263	De Cendana	N/A	800	Selangor - Setia Alam	Apartment	0	0	0	2	Partially Furnished
12	100676391	De Cendana	N/A	850	Selangor - Setia Alam	Apartment	0	0	0	2	Partially Furnished
13	100031707	Soho Suites @ KLCC	2013	15000	Kuala Lumpur - KLCC	Service Residence	3	0	0	3	Fully Furnished
14	99410613	Setapak Ria Condominium	1994	80	Kuala Lumpur - Setapak	Condominium	1	0	0	5	Not Furnished
15	100626230	Pangsapuri Seri Suria (Sunway Ser	N/A	650	Selangor - Kajang	Apartment	0	0	0	7	Partially Furnished
16	100758900	Hijauan Puteri Condominiums	2013	700	Selangor - Puchong	Condominium	0	0	0	13	Fully Furnished
17	99970576	Continew Condominium	2020	200	Kuala Lumpur - KL City	Condominium	1	0	0	20	Not Furnished
18	100303122	Residensi Harmoni 2 @ Bukit Prima	2019	700	Kuala Lumpur - Segambut	Condominium	6	0	0	20	Fully Furnished
19	100627409	N/A	N/A	150	Selangor - Sungai Buloh	Others	0	0	0	25	Not Furnished
20	100810227	BSP21	2018	150	Selangor - Jenjarom	Service Residence	0	0	0	40	Not Furnished
21	100848977	Makmur Apartment	N/A	1200	Selangor - Bandar Sunway	Apartment	0	0	0	50	Partially Furnished
22	99930214	Zen Suites @ Zetapark	N/A	1600	Kuala Lumpur - Setapak	Service Residence	1	0	0	54	Partially Furnished
23	100664088	Canopy Hills	2022	1300	Selangor - Kajang	Condominium	0	0	0	59	Partially Furnished
24	100221594	Southlink	N/A	150	Kuala Lumpur - Bangsar South	Apartment	1	0	0	60	Not Furnished
25	100230309	Platinum Hill PV 5	2006	150	Kuala Lumpur - Setapak	Condominium	1	0	0	80	Not Furnished
26	99774218	Ascenda Residences @ SkyArena Se	2017	550	Kuala Lumpur - Setapak	Condominium	1	0	0	80	Not Furnished
27	98053529	N/A	N/A	580	Selangor - Shah Alam	Apartment	0	0	0	80	Fully Furnished
28	70255817	N/A	N/A	800	Selangor - Bandar Sunway	Apartment	0	0	0	85	Not Furnished

Figure 38: CSV File Output: Heap Sort (Size)

In the output file above, we can see that the "Size" attribute has been sorted according to minimum to maximum values.

3.5.2 CSV File Output (Quick Sort)

ads_id	prop_name	completion_year	monthly_rent (RM)	location	property_type	rooms	parking	bathroom	size (sq.ft.)	furnished
100849472	Affina Bay @ Butterworth	N/A	0	Selangor - 77	Condominium	0	0	0	1500	N/A
100849484	N/A	N/A	0	Selangor - 149	Bungalow House	0	0	0	534	N/A
100549909	One South	2013	70	Selangor - Seri Ker	Service Residence	0	0	0	100	Fully Furnished
98342005	N/A	N/A	80	Selangor - Kota Da	Apartment	0	0	0	400	Fully Furnished
99410613	Setapak Ria Condominium	1994	80	Kuala Lumpur - Se	Condominium	1	0	0	5	Not Furnished
100676485	N/A	N/A	85	Selangor - Seri Ker	Soho	0	0	0	761	N/A
99974016	Idaman Sutera	N/A	90	Kuala Lumpur - Se	Condominium	1	0	0	100	Not Furnished
100213071	Platinum Lake PV 15	N/A	100	Kuala Lumpur - Se	Condominium	4	0	0	1500	Fully Furnished
100183866	Melur Apartment (Sentul)	2006	100	Kuala Lumpur - Se	Apartment	1	0	0	100	Not Furnished
99846469	N/A	N/A	100	Kuala Lumpur - Mi	Residential	0	0	0	100	N/A
99389782	SD Tiara Apartment	N/A	100	Selangor - Bandar	Apartment	0	0	0	400	Fully Furnished
100578461	Kristal Heights (Shah Alam)	2006	100	Selangor - Shah Al	Condominium	0	0	0	1000	Not Furnished
100163744	Platinum Lake PV8	N/A	100	Kuala Lumpur - Se	Condominium	1	0	0	1	Not Furnished
100129853	Platinum Hill PV 2	2013	100	Kuala Lumpur - Se	Condominium	1	0	0	100	Not Furnished
99872270	Platinum Lake PV 15	N/A	100	Kuala Lumpur - Se	Condominium	1	0	0	500	Not Furnished
100255897	Angkasa Condominiums	2005	110	Kuala Lumpur - Ch	Condominium	1	0	0	99999999	Not Furnished
100501147	Bayu Puteri	2004	120	Selangor - Petaling	Apartment	0	0	0	300	Partially Furnished
99974687	Pangsapuri Melur (Sentul)	2006	120	Kuala Lumpur - Se	Apartment	3	0	0	1000	Not Furnished
99955705	Riverville Residences	2015	120	Kuala Lumpur - Ol	Condominium	3	0	0	1350	Fully Furnished
97105324	Maxim Citilights	2017	120	Kuala Lumpur - Se	Service Residence	1	0	0	160	Not Furnished
100694064	The Grand Subang SS13	N/A	120	Selangor - Subang	Service Residence	0	0	0	548	Fully Furnished
98295641	mont kiara pines	N/A	145	Kuala Lumpur - Mo	Condominium	1	0	0	120	Not Furnished
100203686	Residensi Pandanmas 1	N/A	150	Kuala Lumpur - De	Apartment	1	0	0	100	Not Furnished
100839756	Centrestage Designer Suite	N/A	150	Selangor - Petaling	Service Residence	0	0	0	111	Not Furnished
100810227	BSP21	2018	150	Selangor - Jenjaro	Service Residence	0	0	0	40	Not Furnished

Figure 39: CSV File Output: Quick Sort (Monthly Rent)

Figure 39 above shows the output csv file sorted using Quick Sort. As we can see from the Figure above, the "Monthly Rent" attributes have been sorted from the minimum value to maximum value in ascending order.

ads_id	prop_name	completion_year	monthly_rent (RM)	location	property_type	rooms	parking	bathroom	size (sq.ft.)	furnished
100494809	Eclipse Residence @ Pan'ga	2018	1200	Selangor - Cyberjaya	Service Residence	0	0	0	1	Fully Furnished
100618080	Laman Suria (Kajang)	N/A	850	Selangor - Kajang	Apartment	0	0	0	1	Not Furnished
100634656	Magna Ville Condominium	2010	950	Selangor - Selayang	Condominium	0	0	0	1	Not Furnished
100163744	Platinum Lake PV8	N/A	100	Kuala Lumpur - Setapak	Condominium	1	0	0	1	Not Furnished
100653347	Kelana Sentral	N/A	2500	Selangor - Petaling Jaya	Condominium	0	0	0	1	Fully Furnished
99945149	Nidoz Residences	2019	180	Kuala Lumpur - Desa Petali	Condominium	1	0	0	1	Not Furnished
98651627	N/A	N/A	2000	Kuala Lumpur - Cheras	Service Residence	1	0	0	1	Fully Furnished
100756915	Suria Jaya	N/A	1300	Selangor - Shah Alam	Apartment	0	0	0	1	Partially Furnished
100779263	De Cendana	N/A	800	Selangor - Setia Alam	Apartment	0	0	0	2	Partially Furnished
100721981	Mahkota Garden Residence	N/A	1600	Selangor - Bandar Mahkota	Condominium	0	0	0	2	Fully Furnished
100676391	De Cendana	N/A	850	Selangor - Setia Alam	Apartment	0	0	0	2	Partially Furnished
100031707	Soho Suites @ KLCC	2013	15000	Kuala Lumpur - KLCC	Service Residence	3	0	0	3	Fully Furnished
99410613	Setapak Ria Condominium	1994	80	Kuala Lumpur - Setapak	Condominium	1	0	0	5	Not Furnished
100626230	Pangsapuri Seri Suria (Sunw	N/A	650	Selangor - Kajang	Apartment	0	0	0	7	Partially Furnished
100758900	Hijauan Puteri Condominiu	2013	700	Selangor - Puchong	Condominium	0	0	0	13	Fully Furnished
100303122	Residensi Harmoni 2 @ Buki	2019	700	Kuala Lumpur - Segambut	Condominium	6	0	0	20	Fully Furnished
99970576	Continew Condominium	2020	200	Kuala Lumpur - KL City	Condominium	1	0	0	20	Not Furnished
100627409	N/A	N/A	150	Selangor - Sungai Buloh	Others	0	0	0	25	Not Furnished
100810227	BSP21	2018	150	Selangor - Jenjarom	Service Residence	0	0	0	40	Not Furnished
100848977	Makmur Apartment	N/A	1200	Selangor - Bandar Sunway	Apartment	0	0	0	50	Partially Furnished
99930214	Zen Suites @ Zetapark	N/A	1600	Kuala Lumpur - Setapak	Service Residence	1	0	0	54	Partially Furnished
100664088	Canopy Hills	2022	1300	Selangor - Kajang	Condominium	0	0	0	59	Partially Furnished
100221594	Southlink	N/A	150	Kuala Lumpur - Bangsar So	Apartment	1	0	0	60	Not Furnished
99774218	Ascenda Residences @ SkyA	2017	550	Kuala Lumpur - Setapak	Condominium	1	0	0	80	Not Furnished

Figure 40: CSV File Output: Quick Sort (Size)

As Figure 40 shown, we can see an output file sorted for “Size” attribute using Quick Sort. We can see that the “Size” attributes are sorted from minimum to maximum.

3.5.3 Time Complexity Output

Running Test	Time Taken (ms)			
	Heap Sort		Quick Sort	
	Monthly Rent	Size	Monthly Rent	Size
1	397	415	201	226
2	394	422	193	244
3	404	399	198	229
4	409	401	195	230
5	402	404	193	225
Average (ms)	399.6	408.2	196	230.8

Table 13: Test Results

3.5.4 Observation

- Quick Sort outperforms Heap Sort for both “Monthly Rent” and “Size” attributes.
- Quick Sort (Monthly Rent) is the fastest with an average time of 196 milliseconds.
- Heap Sort (Size) is the slowest with an average time of 408.2 milliseconds.
- Quick Sort's efficiency is evident from the results, with a significant performance advantage over Heap Sort.

4.0 Discussion

Based on the test results, Quick Sort has an average-case time complexity of $O(n\ log\ n)$, but it can degrade to $O(n^2)$ in the worst case if poor pivot selection is used. However, with effective strategies such as randomization or median-of-three pivot selection, the average-case $O(n\ log\ n)$ is generally achieved. On the other hand, Heap Sort consistently operates with a time complexity of $O(n\ log\ n)$ in both average and worst-case scenarios. Despite this, the empirical results indicate that Quick Sort's practical performance advantages make it a better choice for this dataset.

The hypothesis was that Quick Sort would perform better than Heap Sort for sorting apartment data based on “Monthly Rent” and “Size”. This hypothesis was grounded in Quick Sort's typically

faster performance due to lower constant factors and better cache performance. The experimental results confirm this hypothesis, showing that Quick Sort is more efficient for the given sorting tasks.

Given the outcomes and its proven better performance, Quick Sort is advised for sorting flat data by "Monthly Rent" and "Size". To maintain optimal performance and avoid worst-case scenarios, it is important to implement effective pivot selection strategies, such as median-of-three or random pivot selection. For very large datasets, where stability (maintaining the order of equal elements) is crucial, Merge Sort could be considered despite its higher memory usage, as it guarantees $O(n\ log\ n)$ time complexity and is stable. Additionally, exploring parallel versions of Quick Sort or other sorting algorithms could leverage multi-core processors to further reduce sorting times. Lastly, always consider the specific use case and dataset characteristics when choosing a sorting algorithm, as certain algorithms may perform better for nearly sorted data or other specific conditions.

5.0 Conclusion

The experimental results indicate that Quick Sort is significantly more efficient than Heap Sort for sorting apartment data by “Monthly Rent” and “Size”, confirming the hypothesis that Quick Sort's lower overhead and better cache efficiency would lead to faster sorting times. While Quick Sort has a worst-case time complexity of $O(n^2)$, practical implementations with effective pivot selection generally achieve $O(n\ log\ n)$ performance, making it the preferred choice for the given dataset. Consequently, Quick Sort is recommended for sorting tasks involving apartment data due to its superior performance. However, it's crucial to implement strategies to mitigate its worst-case scenarios and consider alternative algorithms like Merge Sort for stability in cases with very large datasets.

6.0 Workload Matrix

Component	**Student 1: Ooi Chong Ming**	**Student2: Yeoh Zi Qing Bryan**	**Student 3: Sim Sau Yang**	**Total**
1.0 Introduction	**33.33%**	**33.33%**	**33.33%**	**100%**
2.0 Theoretical Study	**33.33%**	**33.33%**	**33.33%**	**100%**
2.1 Merge Sort	**33.33%**	**33.33%**	**33.33%**	**100%**
2.2 Quick Sort	**33.33%**	**33.33%**	**33.33%**	**100%**
2.3 Heap Sort	**33.33%**	**33.33%**	**33.33%**	**100%**
2.4 Selection Sort	**33.33%**	**33.33%**	**33.33%**	**100%**
2.5 Insertion Sort	**33.33%**	**33.33%**	**33.33%**	**100%**
2.6 Sorting Algorithm Comparison	**33.33%**	**33.33%**	**33.33%**	**100%**
3.0 Comparative Experiment Implementation	**33.33%**	**33.33%**	**33.33%**	**100%**
3.1 Data Review	**33.33%**	**33.33%**	**33.33%**	**100%**
3.2 Building Structure	**33.33%**	**33.33%**	**33.33%**	**100%**
3.3 Reading and Processing Dataset	**33.33%**	**33.33%**	**33.33%**	**100%**
3.4 Sorting Algorithm Execution	**33.33%**	**33.33%**	**33.33%**	**100%**
3.5 Result	**33.33%**	**33.33%**	**33.33%**	**100%**
4.0 Discussion	**33.33%**	**33.33%**	**33.33%**	**100%**
5.0 Conclusion	**33.33%**	**33.33%**	**33.33%**	**100%**

7.0 References

Baeldung. (2024a, January 8). Bubble Sort in Java. https://www.baeldung.com/java-bubble-sort’

Baeldung. (2024b, May 11). Merge Sort in Java. https://www.baeldung.com/java-merge-sort

Code 360 by Coding Ninjas. (n.d.). 2024 Naukri.com. https://www.naukri.com/code360/library/time-and-space-complexities-of-sorting-algorithms-explained

DSA merge sort. (n.d.). W3Schools.com. https://www.w3schools.com/dsa/dsa_algo_mergesort.php

freeCodeCamp.org. (2020, January 25). Bubble Sort explained. https://www.freecodecamp.org/news/bubble-sort/

GeeksforGeeks. (2014, January 31). Selection Sort Algorithm. https://www.geeksforgeeks.org/selection-sort/?ref=lbp

GeeksforGeeks. (2023, May 22). Time and Space complexity analysis of Selection Sort. https://www.geeksforgeeks.org/time-and-space-complexity-analysis-of-selection-sort/

GeeksforGeeks. (2024a, February 8). Time and Space Complexity of Insertion Sort. https://www.geeksforgeeks.org/time-and-space-complexity-of-insertion-sort-algorithm/?ref=header_search

GeeksforGeeks. (2024b, March 18). Time and Space complexity Analysis of bubble sort. https://www.geeksforgeeks.org/time-and-space-complexity-analysis-of-bubble-sort/

GeeksforGeeks. (2024c, March 29). Heap Sort Data Structures and Algorithms Tutorials. https://www.geeksforgeeks.org/heap-sort/

GeeksforGeeks. (2024d, April 12). Bubble Sort data structure and algorithm tutorials. https://www.geeksforgeeks.org/bubble-sort/

GeeksforGeeks. (2024e, June 7). QuickSort Data Structure and Algorithm Tutorials. https://www.geeksforgeeks.org/quick-sort/

GeeksforGeeks. (2024f, May 7). Insertion Sort Data Structure and Algorithm Tutorials. https://www.geeksforgeeks.org/insertion-sort/?ref=shm

GeeksforGeeks. (2024g, May 27). Merge Sort Data Structure and Algorithms tutorials. https://www.geeksforgeeks.org/merge-sort/

GeeksforGeeks. (2024h, April 17). Selection Sort Data Structure and Algorithm Tutorials. https://www.geeksforgeeks.org/selection-sort/?ref=lbp

Heap Sort (With Code in Python, C++, Java and C). (n.d.). https://www.programiz.com/dsa/heap-sort

Heap Sort Algorithm. (n.d.). https://www.tutorialspoint.com/data_structures_algorithms/heap_sort_algorithm.htm

Heap Sort - javatpoint. (n.d.). www.javatpoint.com. https://www.javatpoint.com/heap-sort

Khan Academy. (n.d.). Merge Sort Algorithm Overview (article). https://www.khanacademy.org/computing/computer-science/algorithms/merge-sort/a/overview-of-merge-sort

Moore, K., Wang, G., Kriksunov, A., & Khim, J. (n.d.). Bubble Sort. Brilliant.org. https://brilliant.org/wiki/bubble-sort/

Pankaj. (2022, August 4). Merge Sort Algorithm - Java, C, and Python implementation. DigitalOcean. https://www.digitalocean.com/community/tutorials/merge-sort-algorithm-java-c-python

Programiz. (n.d.-a). Bubble Sort (With code in Python/C++/Java/C). https://www.programiz.com/dsa/bubble-sort

Programiz. (n.d.-b). Merge Sort (With code in Python/C++/Java/C). https://www.programiz.com/dsa/merge-sort

Quick Sort - javatpoint. (n.d.). www.javatpoint.com. https://www.javatpoint.com/quick-sort

QuickSort (With Code in Python/C++/Java/C). (n.d.). https://www.programiz.com/dsa/quick-sort

Selection Sort: Algorithm, Java, C++, Time Complexity. (n.d.). StudySmarter UK. https://www.studysmarter.co.uk/explanations/computer-science/algorithms-in-computer-science/selection-sort/#:~:text=Selection%20Sort%20has%20a%20time

Simplilearn. (2024, March 5). What is Quick Sort Algorithm: How does it Work and its Implementation. Simplilearn.com. https://www.simplilearn.com/tutorials/data-structure-tutorial/quick-sort-algorithm

The Selection Sort Algorithm Explained with Examples. (2019, September 8). The FreeCodeCamp Forum. https://forum.freecodecamp.org/t/the-selection-sort-algorithm-explained-with-examples/16106

Vaia. (n.d.-a). https://app.vaia.com/studyset/18877421/summary/71140938

Vaia. (n.d.-b). https://app.vaia.com/search/smart-explain